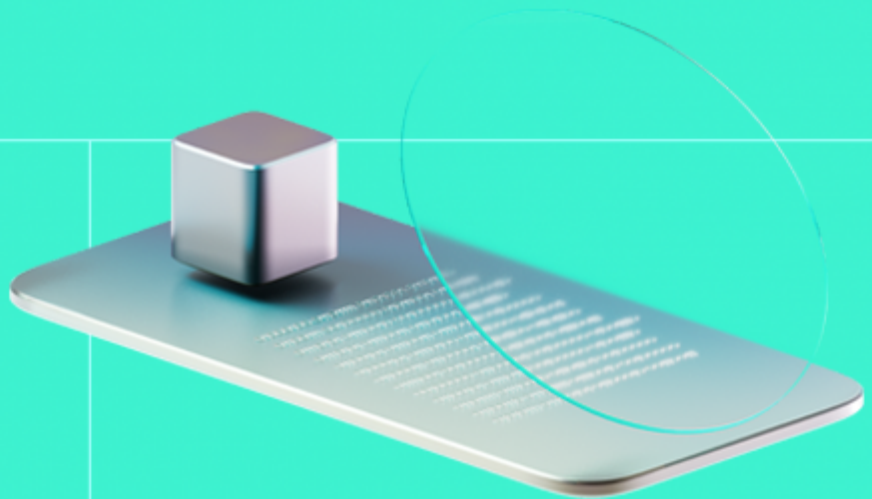




Smart Contract Code Review And Security Analysis Report

Customer: S3 Markets

Date: 06/02/2026



We express our gratitude to the S3markets team for the collaborative engagement that enabled the execution of this Smart Contract Security Assessment.

S3 contracts are smart contracts for environmental attribute certificates (ERC-1155), custody, transfer gating, and paymaster flows.

Document

Name	Smart Contract Code Review and Security Analysis Report for S3markets
Audited By	Turgay Arda Usman, Khrystyna Tkachuk
Approved By	Panagiotis Konstantinidis
Website	https://www.s3markets.com/
Changelog	22/01/2026 - Preliminary Report
	04/02/2026 - Second Report
	06/02/2026 - Final Report
Platform	EVM
Language	Solidity
Tags	ERC1155, Vault, Upgradable
Methodology	https://docs.hacken.io/methodologies/smart-contracts

Review Scope

Repository	https://github.com/S3-Markets/s3block
Initial Commit	1df2037
Final Commit	00fc907

Audit Summary

The system users should acknowledge all the risks summed up in the risks section of the report

19	19	0	0
Total Findings	Resolved	Accepted	Mitigated

Findings by Severity

Severity	Count
Critical	0
High	2
Medium	1
Low	1

Vulnerability	Severity	Status
F-2026-14799 - Funds Loss via Direct Transfer in CustodyVault	High	Fixed
F-2026-14800 - Expired Tokens Permanently Trapped	High	Fixed
F-2026-14811 - Expired Tokens Permanently Locked Due to Burn Restriction	Medium	Fixed
F-2026-14812 - Transfer Gate Bypassed by Non-Allowlisted Operators	Low	Fixed
F-2026-14791 - Floating Pragma	Info	Fixed
F-2026-14792 - Missing Zero Address Validation	Info	Fixed
F-2026-14801 - Missing Array Length Validation	Info	Fixed
F-2026-14802 - Mismatch Between NatSpec Documentation and Implementation	Info	Fixed
F-2026-14803 - Presence of Unused Code	Info	Fixed
F-2026-14804 - Lack of Event Emitting in setToken()	Info	Fixed
F-2026-14805 - Insufficient Role Segregation in CustodyVault	Info	Fixed
F-2026-14806 - Public Function not Used Internally can be Marked External	Info	Fixed
F-2026-14807 - Unnecessary Addition to _buyers in retireFromExternal	Info	Fixed
F-2026-14808 - Unchecked Return Value of _consumeBalancesForRetire	Info	Fixed
F-2026-14809 - Missing Storage Gap in Upgradeable Contract	Info	Fixed
F-2026-14810 - Missing supportsInterface Override	Info	Fixed
F-2026-14813 - Redundant Zero-Address Checks in Transfer Logic	Info	Fixed
F-2026-14814 - Missing Validation Between Token Expiry and Market Entry Period	Info	Fixed
F-2026-14815 - Inactive Buyers Remain in _buyers Indefinitely	Info	Fixed

Documentation quality

- Functional requirements are partially provided.
- Technical description is provided.

Code quality

- The code partially follows best practices and style guides:
 - See informational findings for more details.
- The development environment is configured.

Test coverage

Code coverage of the project is **86.63%** (branch coverage).

- Deployment and basic user interactions are covered with tests.
- Negative cases coverage is present.
- Interactions by several users are not tested.

Table of Contents

System Overview	6
Privileged Roles	6
Potential Risks	7
Findings	8
Vulnerability Details	8
Disclaimers	46
Appendix 1. Definitions	47
Severities	47
Potential Risks	47
Appendix 2. Scope	48
Appendix 3. Additional Valuables	49

System Overview

S3 contracts are smart contracts for environmental attribute certificates (ERC-1155), custody, transfer gating, and paymaster flows. It contains the following contracts:

EAC1155 — ERC-1155 token with supply caps, roles, and transfer gating

CustodyVault — Custody, allocation, and retirement of tokens (the only upgradeable contract)

UUPSProxy — UUPS proxy contract

TransferGate — Allow-list for wallet-to-wallet transfers

Privileged roles

- The `OPS_ROLE` of the `CustodyVault.sol` contract can manage vault operations.
- The `DEFAULT_ADMIN_ROLE` of the `CustodyVault.sol` contract can set a token address.
- The `DEFAULT_ADMIN_ROLE` of the `CustodyVault.sol` contract can grant / revoke all other roles.
- The `ISSUER_ROLE` of the `EAC1155.sol` contract can create token IDs, mint tokens, and set transfer gates.
- The `BURNER_ROLE` of the `EAC1155.sol` contract can burn or retire tokens.
- The `PASUER_ROLE` of the `EAC1155.sol` contract can manage emergency pause mechanism.
- The `GATE_ADMIN` of the `TransferGate.sol` contract can set transfer permissions.
- The `DEFAULT_ADMIN_ROLE` of the `TransferGate.sol` contract can grant / revoke all other roles.

Potential Risks

- **Concentrated minting authority:** The **EAC1155** token design gives a single administrative role broad discretion over minting, including the ability to mint not only to the **CustodyVault** but also to other addresses. While this may be intended for operational flexibility, it increases reliance on internal controls and governance processes to prevent unexpected issuance that could impact token supply expectations and stakeholder confidence.
Dynamic Array Iteration Gas Limit Risks: The project iterates over large dynamic arrays, which leads to excessive gas costs, risking denial of service due to out-of-gas errors, directly impacting contract usability and reliability.
Single Points of Failure and Control: Parts of the system rely on a limited set of privileged roles and operational processes. This concentration can simplify coordination, but it also means overall availability and decision execution may depend on a small number of entities, increasing sensitivity to operational issues or targeted compromise.
Flexibility and Risk in Contract Upgrades: The **CustodyVault** contract is upgradeable, which can be beneficial for maintenance, improvements, and responding to issues. At the same time, upgrades introduce an additional governance and security requirement: users and integrators must rely on the upgrade process being appropriately restricted, reviewed, and transparently managed to reduce the risk of unexpected or unauthorized changes.
Token Metadata and URI Modification Authority: The **EAC1155** admin is able to update token **URIs** after issuance. The project design binds each tokenId at mint time to a cryptographic metadata hash, making any substantive modification to metadata content externally detectable through a hash mismatch. This provides an important integrity signal and limits the ability to alter the semantic meaning of an issued **EAC** without detection. At the same time, URI-level updates still influence how metadata is resolved, displayed, and interpreted by wallets, marketplaces, and other consumers. As a result, changes to hosting, structure, or presentation may affect user perception and integration behavior over time. Clearly defining the intended scope of URI updates and making the hash-binding mechanism easy to observe helps ensure consistent expectations and reduces ambiguity for token holders and integrators.
Coarse-grained Authorization Model Risks: The authorization model grants broad permissions to certain addresses. If any privileged key is compromised, the potential impact could be significant, as an attacker may be able to perform high-impact actions. This makes privilege minimization and strong key management especially important.
Owner's Unrestricted State Modification: The owner has wide latitude to adjust state variables without explicit limitations. This can be useful for administration and responding to evolving needs, but it also increases the importance of transparency, well-defined governance, and change-management practices, particularly around sensitive phases such as minting or distribution, to maintain predictable behavior and user confidence.

Findings

Vulnerability Details

F-2026-14799 - Funds Loss via Direct Transfer in CustodyVault - High

Description:

The `CustodyVault` contract is designed to hold `EAC1155` tokens on behalf of buyers, tracking ownership via internal accounting mappings (`unassigned`, `freeBalance`, `lockedBalance`). Users or admins deposit tokens into the Vault, which can then be allocated, transferred internally, locked, or retired. The contract implements `IERC1155Receiver` to accept token transfers.

The `CustodyVault` accepts `ERC1155` tokens sent via direct transfers (`safeTransferFrom()`) but fails to account for them in its internal ledgers. Consequently, any tokens sent directly to the Vault by users are permanently locked in the contract, as they are not credited to any `freeBalance` or `unassigned` supply, rendering them impossible to withdraw or utilize.

The issuerability lies in the implementation of `onERC1155Received()` and `onERC1155BatchReceived()` within `CustodyVault.sol`. These functions are triggered whenever the Vault receives `ERC1155` tokens. In the current implementation, the code only updates the internal unassigned balance if the sender (`from`) is the zero address, which indicates a minting:

```
function onERC1155Received(
    address,
    address from,
    uint256 id,
    uint256 value,
    bytes calldata
) external override returns (bytes4) {
    require(msg.sender == address(token), "Vault: token only");
    // If sender is zero address (minting), mark as unassigned
    if (from == address(0)) {
        unassigned[id] += value;
        emit ICustodyVault.InventoryReceived(id, value, bytes32(0));
    }

    return IERC1155Receiver.onERC1155Received.selector;
}
```

If a user calls `safeTransferFrom()` on the token contract to send funds to the Vault (a standard `ERC1155` interaction), the token contract transfers the

balance to the Vault and calls this hook. The hook returns the valid selector, so the transaction succeeds. However, because from is the user's address (not `address(0)`), the if block is skipped. The Vault physically holds the tokens, but the `freeBalance` for the user remains 0.

Since withdrawal (`withdrawToExternal()`) and usage (`retire()`, `internalTransfer()`) strictly require `freeBalance[buyerHash][id] >= amount`, these tokens become inaccessible. This leads to situations where tokens are locked in the Vault contract address indefinitely.

Assets:

- CustodyVault.sol [<https://github.com/S3-Markets/s3block>]

Status:

Fixed

Classification

Impact Rate:	5/5
Likelihood Rate:	4/5
Exploitability:	Independent
Complexity:	Simple
Severity:	High

Recommendations

Remediation: If direct transfers must be supported, the data parameter should be decoded to retrieve the target `buyerHash`, and the `freeBalance` should be updated accordingly. However, reverting is safer to ensure correct usage of the dedicated `depositFor()` flow.

Resolution: In commit `b318424`, an additional validation has been added in the `onERC1155Received` function to prevent direct transfers to the `CustodyVault` contract. The `operator` parameter is now required to be the `CustodyVault` contract itself, enforcing the use of the `depositFor()` function, which ensures correct token balance accounting.

Evidences

PoC

Reproduce:

1. The user (externalWallet) holds 50 tokens (e.g., Token ID 1) in their own wallet, and the Vault has a balance of 0 for this token.

2. The user calls `token.safeTransferFrom` to send these 50 tokens directly to the CustodyVault address. This is a standard ERC1155 transfer operation, bypassing the Vault's specific `depositFor` function.
3. The token contract calls the Vault's `onERC1155Received` hook. The Vault returns the success selector because the code only explicitly checks if the sender is `address(token)`, which it is. However, the logic inside the hook skips updating any balances because the `from` address is not `address(0)` (it is the user's address).

PoC Code:

```
function testFundLossViaDirectTransfer() public {
    _allowListExternalAndVault();

    uint256 transferAmount = 50;
    vm.startPrank(admin);
    token.mintTo(externalWallet, tokenId, transferAmount, "");
    vm.stopPrank();

    assertEq(token.balanceOf(externalWallet, tokenId), transferAmount);
    assertEq(token.balanceOf(address(vault), tokenId), 0);
    assertEq(vault.unassigned(tokenId), 0);

    vm.startPrank(externalWallet);
    token.safeTransferFrom(externalWallet, address(vault), tokenId, transferAmount, "");
    vm.stopPrank();

    assertEq(token.balanceOf(address(vault), tokenId), transferAmount);

    assertEq(vault.unassigned(tokenId), 0, "Unassigned balance should not increase on direct transfer");

    bytes32 senderHash = keccak256(abi.encodePacked(externalWallet));
    assertEq(vault.freeBalance(senderHash, tokenId), 0, "Sender free balance should be 0");

    // Check arbitrary buyer hash (just in case)
    assertEq(vault.freeBalance(buyerHash, tokenId), 0, "Arbitrary buyer free balance should be 0");
}
```

Results:

```
Ran 1 test for test/CustodyVault.t.sol:CustodyVaultTest
[PASS] testFundLossViaDirectTransfer() (gas: 199153)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 8.27ms (753.08µs CPU time)
```

F-2026-14800 - Expired Tokens Permanently Trapped - High

Description:

The `CustodyVault` contract acts as a centralized custodian for `EAC1155` tokens, maintaining internal ledgers (`freeBalance`, `lockedBalance`) for buyers identified by hashes. The vault manages the lifecycle of these credits, including allocation, internal transfers, locking for claims, retirement, and withdrawal to external wallets. Most state-changing operations are restricted to the `OPS_ROLE`.

A logic flaw in the `CustodyVault` prevents any action on expired tokens, causing them to become permanently trapped in the vault. Strict expiry checks in all state-changing functions mean that once a token reaches its expiration timestamp, it cannot be withdrawn, retired, unlocked, or moved, effectively freezing the associated storage slots and preventing any administrative cleanup.

The `CustodyVault` enforces a strict check `require(!token.isExpired(id), "Vault: token has expired")` at the beginning of nearly every function that interacts with token balances. This includes:

- `withdrawToExternal()`
- `retire()` (and `retireBatch()`, `retireBatchSameReason()`, `retireFromExternal()`)
- `unlock()`
- `internalTransfer()`

This creates a deadlock scenario for assets held within the vault. If a user holds tokens in `freeBalance` or `lockedBalance` and the token expires:

1. **Retirement is blocked:** Users cannot burn the tokens to remove them from circulation or claim them (e.g., for an "expired" classification). The retire function reverts immediately.
2. **Withdrawal is blocked:** Users cannot withdraw the tokens to their own wallet (`withdrawToExternal()` reverts).
3. **Unlocking is blocked:** If tokens were locked via `lockForClaim()` (e.g., pending a claim verification that took too long), they cannot be unlocked back to `freeBalance` (unlock reverts).

```
function retire(
    bytes32 buyerHash,
    uint256 id,
    ...
) external onlyRole(OPS_ROLE) {
    require(!token.isExpired(id), "Vault: token has expired"); // <--- Pre
vents cleanup
    ...
    token.burn(address(this), id, amount, reason);
```

```

    ...
}

function withdrawToExternal(
    bytes32 buyerHash,
    uint256 id,
    ...
) external onlyRole(OPS_ROLE) {
    require(!token.isExpired(id), "Vault: token has expired"); // <--- Prevents withdrawal
    ...
}

```

This effectively turns the vault into a "trap" where tokens can enter but can never leave or be destroyed once they expire. Hence, user balances for expired tokens are permanently "bricked." They cannot be utilized, claimed, or removed.

Assets:

- CustodyVault.sol [<https://github.com/S3-Markets/s3block>]

Status:

Fixed

Classification

Impact Rate:	5/5
Likelihood Rate:	3/5
Exploitability:	Independent
Complexity:	Simple
Severity:	High

Recommendations

Remediation: Remove the `require(!token.isExpired(id))` check from the `retire` family of functions. Retirement is the standard way to handle expired or used assets. Ensure the underlying `EAC1155.burn()` function also permits burning expired tokens.

Resolution: In commit `ee718b2`, the `retire` (or `withdrawToExternal`) function still enforces a direct restriction based on token expiration, but new functions have been introduced to handle expired token utilization. The `resolveExpired()` and `resolveExpiredBatch()` functions allow the `OPS_ROLE` to burn expired tokens without accounting for them in `retiredBalance`, `totalRetiredByTokenId`, or the

`_retiredBuyersForToken` mappings. This effectively serves as a cleanup mechanism for expired tokens.

Evidences

PoC

Reproduce:

1. A token is created with an expiry date.
2. Tokens are minted to the Vault and allocated to a buyer.
3. Some tokens are locked for a claim.
4. Time is warped past the token's expiry.
5. All attempts to interact with the tokens (`withdrawToExternal`, `retire`, `unlock`, `internalTransfer`) fail with the revert reason "Vault: token has expired", proving the assets are frozen.

PoC Code:

```
function testExpiredTokensPermanentlyTrapped() public {
    // Setup
    uint256 expiry = block.timestamp + 10 days;
    uint256 testTokenId = createToken(1000, expiry);

    // Mint tokens to the vault and allocate them
    vm.prank(admin);
    token.mintTo(address(vault), testTokenId, 100, "");

    vm.startPrank(ops);
    vault.allocate(buyerHash, testTokenId, 100, "ref");

    // User locks some tokens for a claim
    vault.lockForClaim(buyerHash, testTokenId, 50, "lock");
    vm.stopPrank();

    // Verify state before expiry
    assertEq(vault.freeBalance(buyerHash, testTokenId), 50);
    assertEq(vault.lockedBalance(buyerHash, testTokenId), 50);

    vm.warp(expiry + 1);

    vm.startPrank(ops);

    vm.expectRevert("Vault: token has expired");
    vault.withdrawToExternal(buyerHash, testTokenId, 10, wallet);

    vm.expectRevert("Vault: token has expired");
    vault.retire(
        buyerHash,
```

```

testTokenId,
10,
bytes32("evidence"),
"uri",
block.timestamp - 2 days,
block.timestamp - 1 days,
RetirementReason.Other
);

vm.expectRevert("Vault: token has expired");
vault.unlock(buyerHash, testTokenId, 50, "unlock");

bytes32 buyer2 = keccak256("buyer2");
vm.expectRevert("Vault: token has expired");
vault.internalTransfer(buyerHash, buyer2, testTokenId, 10, "transfer");

vm.stopPrank();
}

```

Results:

```

Ran 1 test for test/CustodyVault.t.sol:CustodyVaultTest
[PASS] testExpiredTokensPermanentlyTrapped() (gas: 436900)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 8.59ms (1.07ms CPU time)

```

F-2026-14811 - Expired Tokens Permanently Locked Due to Burn Restriction - Medium

Description:

The `EAC1155` contract manages Environmental Attribute Certificates that have an expiration date. The contract includes burn and burnBatch functions, restricted to the `BURNER_ROLE`, to retire certificates or remove them from circulation.

Tokens that reach their expiry timestamp become permanently locked in user wallets because the burn functions enforce an expiry check. This prevents even authorized roles from removing expired inventory, resulting in "zombie" tokens that clutter the ledger and cannot be cleaned up. The `burn()` and `burnBatch()` functions explicitly call `_checkExpiry(id)` before executing the burn logic. This modifier reverts the transaction if `block.timestamp > expiry`.

```
function burn(address from, uint256 id, uint256 amount, RetirementReason reason)
on)
    external
    onlyRole(BURNER_ROLE)
    whenNotPaused
    {
        _checkTokenIsCreated(id);
        _checkExpiry(id);
        _burn(from, id, amount);
        emit IEAC1155.TokenBurned(from, id, amount, reason, msg.sender);
    }
```

While preventing transfers of expired tokens is correct behavior for valid certificates, preventing burning creates a deadlock. Once a token expires, no action can ever be taken on it again. This can lead to situations where.

- Users and the protocol cannot clean up expired inventory. Wallets will permanently display invalid certificates.
- The contract state cannot be reduced by burning dead tokens.

Assets:

- EAC1155.sol [<https://github.com/S3-Markets/s3block>]

Status:

Fixed

Classification

Impact Rate:

3/5

Likelihood Rate:	3/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Medium

Recommendations

Remediation: Remove the `_checkExpiry(id)` check from the `burn()` and `burnBatch()` functions, or allow a specific `RetirementReason.Expired` to bypass this check. This allows the `BURNER_ROLE` to remove invalid/expired tokens from circulation.

Resolution: In commit `3b40377`, the `burn` functions perform a validation to check if the `RetirementReason` is set as `Expired`; if so, the `_checkExpiry(ids[i])` step is skipped, meaning the burning is now allowed for the expired tokens.

Evidences

PoC

Reproduce:

1. Create a token with a short expiry.
2. Mint tokens to a user.
3. Fast-forward time past the expiry.
4. Verify the user cannot transfer the tokens.
5. Verify the authorized burner cannot burn the tokens, confirming the tokens are stuck in the "Zombie" state.

PoC Code:

```
function testZombieTokenLockIn() public {
    //Create a token that expires soon
    uint256 expiry = block.timestamp + 1 days;
    uint256 tokenId = createToken(1000, expiry);

    //Mint tokens to user1
    vm.prank(issuer);
    token.mint(user1, tokenId, 100, "");

    //Fast forward past expiry
    vm.warp(expiry + 1);

    //Verify user1 still has balance
    assertEq(token.balanceOf(user1, tokenId), 100);
}
```



```
//Attempt to transfer
vm.prank(user1);
vm.expectRevert("EAC1155: token has expired");
token.safeTransferFrom(user1, user2, tokenId, 10, "");

//Attempt to burn/retire
vm.prank(burner);
vm.expectRevert("EAC1155: token has expired");
token.burn(user1, tokenId, 100, RetirementReason.Other);

//Verify balance remains stuck
assertEq(token.balanceOf(user1, tokenId), 100);
}
```

F-2026-14812 - Transfer Gate Bypassed by Non-Allowlisted Operators - Low

Description:

The `EAC1155` contract integrates with a `TransferGate` to enforce allowlists for transfers. The gate is intended to ensure that only authorized entities can hold and transfer tokens, likely for compliance (KYC/AML) reasons.

The transfer gate logic verifies that the `from` (sender) and `to` (recipient) addresses are allowed, but it fails to check the status of the `msg.sender` (operator). This allows a banned entity to continue operating as a transaction relayer for other users, bypassing the spirit of a strict compliance blacklist. In `safeTransferFrom()`, the gate check is applied as follows:

```
require(
    address(transferGate) != address(0) ? transferGate.isTransferAllowed(from
, to, id, value) : true,
    "EAC1155: transfer blocked by gate"
);
```

The `TransferGate.isTransferAllowed()` function only checks `allowed[from]` and `allowed[to]`.

```
function isTransferAllowed(address from, address to, ...) external view returns (bool) {
    return allowed[from] && allowed[to];
}
```

If a malicious or sanctioned entity is removed from the allowlist, they cannot hold tokens (to check fails) or send their own tokens (from check fails). However, if an allowed party has previously approved removed address as an operator, the removed address can call `safeTransferFrom()` and the transaction will succeed. This will create a compliance gap since a sanctioned entity can still actively participate in the protocol by facilitating transfers between other users.

Assets:

- `EAC1155.sol` [<https://github.com/S3-Markets/s3block>]

Status:

Fixed

Classification

Impact Rate:

2/5

Likelihood Rate: 2/5

Exploitability: Semi-Dependent

Complexity: Simple

Severity: Low

Recommendations

Remediation: Update the `TransferGate` interface and implementation to accept and check the operator (`msg.sender`).

Resolution: In commit `6d06f6c`, the `msg.sender` is now also required to be whitelisted for transfer.

F-2026-14791 - Floating Pragma - Info

Description:

In Solidity development, the pragma directive specifies the compiler version to be used, ensuring consistent compilation and reducing the risk of issues caused by version changes. However, using a floating pragma (e.g., `^0.8`) introduces uncertainty, as it allows contracts to be compiled with any version within a specified range. This can result in discrepancies between the compiler used in testing and the one used in deployment, increasing the likelihood of vulnerabilities or unexpected behavior due to changes in compiler versions.

The project currently uses floating pragma declarations (`^0.8`) in its Solidity contracts. This increases the risk of deploying with a compiler version different from the one tested, potentially reintroducing known bugs from older versions or causing unexpected behavior with newer versions. These inconsistencies could result in security vulnerabilities, system instability, or financial loss. Locking the pragma version to a specific, tested version is essential to prevent these risks and ensure consistent contract behavior.

Assets:

- CustodyVault.sol [<https://github.com/S3-Markets/s3block>]
- EAC1155.sol [<https://github.com/S3-Markets/s3block>]
- TransferGate.sol [<https://github.com/S3-Markets/s3block>]
- UUPSPProxy.sol [<https://github.com/S3-Markets/s3block>]

Status:

Fixed

Classification

Impact Rate:	1/5
Likelihood Rate:	1/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Info

Recommendations

Remediation:

It is recommended to **lock the pragma version** to the specific version that was used during development and testing. This ensures that the contract will always be compiled with a known, stable compiler version, preventing unexpected changes in behavior due to compiler updates.

Before selecting a version, review known bugs and vulnerabilities associated with each Solidity compiler release. This can be done by referencing the official Solidity compiler release notes: [Solidity GitHub releases](#) or [Solidity Bugs by Version](#). Choose a compiler version with a good track record for stability and security.

Resolution:

Fixed in `9108609`, the compiler version was locked to `0.8.33`.

F-2026-14792 - Missing Zero Address Validation - Info

Description:

In Solidity, the Ethereum address `0x0000000000000000000000000000000000000000000000000000000000000000` is known as the **zero address**. This address has significance because it is the default value for uninitialized address variables and is often used to represent an invalid or non-existent address.

The issue arises when a Solidity smart contract does not properly check or prevent interactions with the zero address, leading to unintended behavior. For instance, a contract might allow tokens to be sent to the zero address without any checks, which essentially burns those tokens as they become irretrievable. While sometimes this is intentional, without proper control or checks, accidental transfers could occur.

Missing checks were observed in the following functions:

- `setAllowed()`
- `constructor()`
- `setTransferGate()`
- `setToken()`

Assets:

- EAC1155.sol [<https://github.com/S3-Markets/s3block>]
- TransferGate.sol [<https://github.com/S3-Markets/s3block>]
- UUPSPProxy.sol [<https://github.com/S3-Markets/s3block>]

Status:

Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 1/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation:

It is strongly recommended to implement checks to prevent the zero address from being set. This can be achieved by adding `require` statements that ensure address parameters are not the zero address.

Resolution:

Fixed in `e408fc4`, the validation for zero address check was implemented in the mentioned functions.

F-2026-14801 - Missing Array Length Validation - Info

Description:

The `onERC1155BatchReceived()` function is a hook called by the ERC1155 token contract when a batch of tokens is transferred or minted to the `CustodyVault`. It updates the unassigned balance for each token ID received.

The `onERC1155BatchReceived()` function assumes the `ids` and `values` arrays have equal lengths without explicitly validating this condition. While the standard ERC1155 implementation guarantees this, relying on external contract behavior without validation is unsafe practice.

The function iterates based on `ids.length` and accesses `values[i]`. If a non-compliant or malicious token contract calls this function with mismatched array lengths (specifically if `values` is shorter than `ids`), it would cause an out-of-bounds read and revert the transaction.

```
for (uint256 i = 0; i < ids.length; i++) {
    unassigned[ids[i]] += values[i];
    ...
}
```

This can cause reverts in the on-received checks.

Assets:

- CustodyVault.sol [<https://github.com/S3-Markets/s3block>]

Status:

Fixed

Classification

Impact Rate:	2/5
Likelihood Rate:	1/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Info

Recommendations

Remediation:

Add a requirement to validate array lengths match at the beginning of the function.

Resolution:

Fixed in [e9034eb](#), array length validation was added in the `onERC1155BatchReceived()` function:

```
require(ids.length == values.length, "Vault: arrays length mismatch");
```

F-2026-14802 - Mismatch Between NatSpec Documentation and Implementation - Info

Description:

The `EAC1155.sol` is an ERC-1155 token contract used to represent Market Environmental Attribute Certificates. Prior to minting, the issuer is required to initialize tokenId-specific metadata via the `createTokenId()` function. One of the required parameters is `expiry`, which defines the expiration timestamp of a given `tokenId`.

According to the NatSpec documentation, the `expiry` parameter is optional. A value of `0` is explicitly documented as indicating that the token has no expiration, while any non-zero value must represent a timestamp in the future:

```
/// @param expiry Expiry timestamp for the token (0 for no expiry, must be in future if set)
```

However, the current implementation enforces the following validation:

```
require(expiry > 0 && expiry > block.timestamp, "EAC1155: expiry must be in future");
```

This condition explicitly disallows `expiry == 0`, thereby preventing the creation of new `tokenId` without an expiration date. As a result, the contract behavior does not match its documented interface and intended design.

Assets:

- `EAC1155.sol` [<https://github.com/S3-Markets/s3block>]

Status:

Fixed

Classification

Impact Rate:	1/5
Likelihood Rate:	1/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Info

Recommendations

Remediation:

Consider updating the validation logic to reflect the documented behavior by allowing `expiry == 0` and enforcing the "future timestamp" constraint only when a non-zero expiry is provided.

A compliant validation check would be:

```
require(  
  expiry == 0 || expiry > block.timestamp,  
  "EAC1155: expiry must be in the future if set"  
);
```

Alternatively, if non-expiring tokenIds are not intended to be supported, the NatSpec documentation should be updated accordingly to clearly state that `expiry` is mandatory and must always be a future timestamp.

Resolution:

Fixed in `20ab678`, the NatSpec was fixed to align with the code implementation. The expiry timestamp can't be `0` value.

```
/// @param expiry Expiry timestamp for the token (must be > 0 and in the future)
```

F-2026-14803 - Presence of Unused Code - Info

Description:

The codebase contains redundant declarations that are never referenced or executed. Specifically:

- The `EAC1155.sol` declares the constant `C02E_DECIMALS` but it is not used anywhere in the contract logic.

```
uint8 public constant C02E_DECIMALS = 3;
```

- The `CustodyVault.sol` defines the internal function `_validateExternalBatchRetirement()` but it is never invoked within the codebase:

```
function _validateExternalBatchRetirement(uint256[] calldata ids, uint256[] calldata amounts) internal view {
    for (uint256 i = 0; i < ids.length; i++) {
        require(!token.isExpired(ids[i]), "Vault: token has expired");
        require(amounts[i] > 0, "Vault: zero amount");
    }
}
```

These elements do not contribute to the contract's behavior and remain unused throughout the codebase.

Unused code contributes to unnecessary bytecode size, potentially increasing deployment and execution costs. It can also reduce code clarity and maintainability over time. While the impact may be minor, it contributes to reduced optimization and could be misleading to users and future contributors.

Assets:

- CustodyVault.sol [<https://github.com/S3-Markets/s3block>]
- EAC1155.sol [<https://github.com/S3-Markets/s3block>]

Status:

Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 2/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: Consider removing unused code from the affected contracts to keep the codebase clean, optimized, and easier to maintain. If these declarations are intended for future use, consider documenting this explicitly or implementing their usage to avoid confusion and improve long-term maintainability.

Resolution: Fixed in `0888e36`, unused code was removed.

F-2026-14804 - Lack of Event Emitting in setToken() - Info

Description:

The `setToken()` function in the `CustodyVault` contract allows an account with the `DEFAULT_ADMIN_ROLE` to update the address of the `EAC1155` token associated with the vault. This operation modifies a critical configuration parameter that directly affects vault behavior.

```
function setToken(address tokenAddress) external onlyRole(DEFAULT_ADMIN_ROLE)
{
    token = IEAC1155(tokenAddress);
}
```

However, no event is emitted when this state change occurs, making the update invisible to off-chain consumers. Events are essential for transparency, monitoring, and debugging, especially for admin actions that affect core protocol configuration and system-level dependencies.

Assets:

- CustodyVault.sol [<https://github.com/S3-Markets/s3block>]

Status:

Fixed

Classification

Impact Rate:	1/5
Likelihood Rate:	2/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Info

Recommendations

Remediation:

Consider emitting a dedicated event within `setToken()` that records the newly set token address. This will improve traceability, facilitate off-chain monitoring, and align the contract with best practices for administrative state changes.

Resolution:

Fixed in `19ac459`, the event `TokenSet()` is now emitting within the `setToken()` function.

F-2026-14805 - Insufficient Role Segregation in CustodyVault - Info

Description:

The `CustodyVault` contract implements both `DEFAULT_ADMIN_ROLE` and `OPS_ROLE`. While `OPS_ROLE` is intended to handle operational responsibilities, the `DEFAULT_ADMIN_ROLE` still retains authority to execute the `setToken()` function, which updates the address of the associated `EAC1155` token.

```
function setToken(address tokenAddress) external onlyRole(DEFAULT_ADMIN_ROLE)
{
    token = IEAC1155(tokenAddress);
}
```

Granting such configuration privileges to `DEFAULT_ADMIN_ROLE` broadens its scope beyond role management and introduces unnecessary risk when the role is used for day-to-day administrative operations. This design weakens the intended separation between governance-level control and routine operational actions.

If the private key controlling the `DEFAULT_ADMIN_ROLE` is compromised, an attacker could gain unauthorized access to critical protocol configuration, including modifying the vault's associated token. This may result in unauthorized configuration changes, disruption of system behavior, or complete control over core protocol parameters. The absence of strict role segregation increases the blast radius of a single compromised role.

Assets:

- CustodyVault.sol [<https://github.com/S3-Markets/s3block>]

Status:

Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 2/5

Exploitability: Dependent

Complexity: Simple

Severity: Info

Recommendations

Remediation:

Introduce stricter role separation by assigning the `setToken()` function exclusively to a dedicated `OPS_ROLE`, and reserve the `DEFAULT_ADMIN_ROLE` solely for role administration (granting, revoking, and managing roles).

Resolution:

Fixed in `19ac459`, the access control for `setToken()` function was adjusted to allow execution only by `OPS_ROLE`.

F-2026-14806 - Public Function not Used Internally can be Marked External - Info

Description:

There are some functions within the protocol codebase that are currently marked as `public`, but are not invoked internally within the contract itself. Since these function is only intended to be called externally, it should be marked as `external` instead of `public`.

Affected functions:

- `EAC1155::getTokenData()`
- `CustodyVault::initialize()`

In Solidity, functions that are only meant to be invoked by external actors, such as external contracts or users, should be designated as `external` rather than `public`. While both `public` and `external` functions are accessible from outside the contract, there are key differences. A `public` functions can be called both from other functions inside the same contract and from outside transactions, while an `external` function can only be called from outside the contract. Specifically, `external` functions are more gas-efficient when called externally, as they do not require the extra internal checks that `public` functions do. Marking a function as `public` instead of `external` introduces unnecessary gas costs.

Assets:

- CustodyVault.sol [<https://github.com/S3-Markets/s3block>]
- EAC1155.sol [<https://github.com/S3-Markets/s3block>]

Status:

Fixed

Classification

Impact Rate:	1/5
Likelihood Rate:	2/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Info

Recommendations

Remediation:

Consider changing visibility to `external` instead of `public` to enhance gas efficiency and reduce unnecessary exposure for the specified functions. This change ensures that the function is only callable from external sources, reducing internal contract complexity and improving security.

Resolution:

Fixed in `1facba9`, the functions visibility was changed to `external`.

F-2026-14807 - Unnecessary Addition to `_buyers` in `retireFromExternal` - Info

Description: The `retireFromExternal()` function allows a user to transfer tokens from their external wallet to the vault and immediately burn them in a single transaction.

The function adds the caller (`msg.sender`) to the `_buyers` registry even though they do not hold any persistent balance in the Vault. This pollutes the buyer registry with transient users who merely used the Vault as a pass-through for burning. The `_buyers` set is intended to track entities that have custody balances (`freeBalance` or `lockedBalance`) within the Vault. In `retireFromExternal()`, the tokens are burned in the same transaction they are received.

```
_buyers.add(buyerHash);
```

The user never holds a balance that requires tracking. Hence this will cause unnecessary gas consumption.

Assets:

- CustodyVault.sol [<https://github.com/S3-Markets/s3block>]

Status:

Fixed

Classification

Impact Rate: 1/5
Likelihood Rate: 1/5
Exploitability: Independent
Complexity: Simple
Severity: Info

Recommendations

Remediation: Remove the line `_buyers.add(buyerHash)` from the `retireFromExternal()` function.

Resolution: Fixed in `8b9c8f7`, the line `_buyers.add(buyerHash)` was removed, buyer is not added to `_buyers` registry on external retirement.

[F-2026-14808](#) - Unchecked Return Value of `_consumeBalancesForRetire` - Info

Description:

The `retire()` function allows a buyer to burn their tokens. It calls the internal helper `_consumeBalancesForRetire()` to deduct the required amount from either the buyer's `lockedBalance`, `freeBalance`, or a combination of both.

The retire function calls `_consumeBalancesForRetire()` but ignores its return values (`fromLocked`, `fromFree`). While the logic inside the helper correctly updates state, ignoring return values can hide logic errors and makes the code harder to reason about and maintain. The helper function returns the breakdown of where funds were taken from:

```
_consumeBalancesForRetire(buyerHash, id, amount);
```

However, the definition explicitly returns data:

```
function _consumeBalancesForRetire(...) internal returns (uint256 fromLocked, uint256 fromFree)
```

This reduces clarity and unused return values increase gas slightly.

Assets:

- CustodyVault.sol [<https://github.com/S3-Markets/s3block>]

Status:

Fixed

Classification

Impact Rate:	1/5
Likelihood Rate:	1/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Info

Recommendations

Remediation:

Either use the return values or remove the return declaration from the `_consumeBalancesForRetire()` function definition if they are truly unneeded.

Resolution:

In commit `a4aaa6f`, the unused return data is removed from `_consumeBalancesForRetire` function.

F-2026-14809 - Missing Storage Gap in Upgradeable Contract - Info

Description:

When working with upgradeable contracts, it is necessary to introduce storage gaps to allow for storage extension during upgrades. Storage gaps are a convention for reserving storage slots in a base contract, allowing future versions of that contract to use up those slots without affecting the storage layout of child contracts.

Without a storage gap, the variable in the child contract might be overwritten by the upgraded base contract if new variables are added to the base contract. This could have unintended and very serious consequences for the child contracts.

Assets:

- CustodyVault.sol [<https://github.com/S3-Markets/s3block>]

Status:

Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 1/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: Introduce Storage Gaps in the affected contracts.

Resolution: In commit `826d3d3`, the `uint256[50] private __gap` is introduced.

F-2026-14810 - Missing supportsInterface Override - Info

Description:

`CustodyVault` inherits from `AccessControlEnumerableUpgradeable` and `IERC1155Receiver`. Both of these parent contracts/interfaces relate to `ERC165` `supportsInterface` checks. The contract does not override `supportsInterface` to include `IERC1155Receiver`. This means calls to `supportsInterface(type(IERC1155Receiver).interfaceId)` will return false, potentially causing compliant `ERC1155` token contracts (which check for receiver support) to revert when sending tokens to the Vault.

Thus, token transfers (specifically `safeTransferFrom()` and `safeBatchTransferFrom()`) from strict `ERC1155` implementations may fail because the Vault claims it does not support the `IERC1155Receiver` interface, causing a denial of service for deposits.

Assets:

- `CustodyVault.sol` [<https://github.com/S3-Markets/s3block>]

Status:

Fixed

Classification

Impact Rate:	1/5
Likelihood Rate:	1/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Info

Recommendations

Remediation: Implement the `supportsInterface` override.

Resolution: In commit `d99c440`, the `supportsInterface()` override is introduced to `CustodyVault`.

F-2026-14813 - Redundant Zero-Address Checks in Transfer Logic - Info

Description:

The `safeTransferFrom()` and `safeBatchTransferFrom()` functions in `EAC1155` contain custom logic to skip the transfer gate if the operation is a mint or burn.

The check `if (from == address(0) ...)` inside the public `safeTransferFrom()` function is unreachable dead code. Standard `ERC1155` logic prevents external callers from initiating a transfer where `from` is the zero address.

```
function safeTransferFrom(address from, address to, ...) public override {
    if (from == address(0) || to == address(0)) {
        super.safeTransferFrom(from, to, id, value, data);
        return;
    }
    ...
}
```

- `from == address(0)`: This case is impossible for external calls. The `super.safeTransferFrom()` (OpenZeppelin implementation) calls `_safeTransferFrom()`, which verifies that `msg.sender` is either from or an approved operator. Since `msg.sender` cannot be `address(0)` and `address(0)` cannot approve operators, from can never be zero.
- `to == address(0)`: While possible to call, OpenZeppelin's `safeTransferFrom()` reverts with `ERC1155InvalidReceiver(address(0))` immediately. Skipping the gate to call super just leads to a standard revert instead of a gate revert.

Assets:

- `EAC1155.sol` [<https://github.com/S3-Markets/s3block>]

Status:

Fixed

Classification

Impact Rate:	1/5
Likelihood Rate:	1/5
Exploitability:	Independent
Complexity:	Simple

Severity:

Info

Recommendations**Remediation:**

Remove the `from == address(0)` check as it is dead code. The `to == address(0)` check can be kept if the intention is to let the standard `ERC1155` error handle burns, but typically burns should be routed through the dedicated burn function, not `safeTransferFrom()`.

Resolution:

In commit `08b3190`, the `from == address(0)` check was removed from the `safeTransferFrom` function.

F-2026-14814 - Missing Validation Between Token Expiry and Market Entry Period - Info

Description:

The `createTokenId()` function initializes a new Environmental Attribute Certificate (EAC) with specific metadata, including an expiry timestamp and a "market entry" window (`market_entry_start` and `market_entry_end`). The market entry window typically represents the reporting period during which the underlying commodity was sold or produced.

The contract fails to enforce a logical relationship between the token's expiration date and its market entry period. Specifically, it allows creating a token that expires before the market entry period has concluded. This could lead to the issuance of certificates that become invalid (`expired`) before their underlying reporting period is even complete, rendering them useless or misleading.

In `createTokenId()`, the contract validates that expiry is in the future and that `market_entry_start <= market_entry_end`. However, it does not check if expiry occurs after `market_entry_end`.

```
function createTokenId(...) external onlyRole(ISSUER_ROLE) returns (uint256)
{
    ...
    require(expiry > 0 && expiry > block.timestamp, "EAC1155: expiry must be
in future");
    require(market_entry_start <= market_entry_end, "EAC1155: market entry st
art must be <= end");
    ...
}
```

If an issuer accidentally sets `expiry < market_entry_end`, the token might expire while the commodity is theoretically still entering the market. For example, a certificate covering sales through `December 31st, 2025`, should not expire on `June 1st, 2025`.

This can lead to:

- Certificates may be issued with contradictory validity periods.
- Issuers might create unusable tokens that expire prematurely, requiring re-issuance and causing confusion.

Assets:

- EAC1155.sol [<https://github.com/S3-Markets/s3block>]

Status:

Fixed

Classification

Impact Rate:	2/5
Likelihood Rate:	1/5
Exploitability:	Semi-Dependent
Complexity:	Simple
Severity:	Info

Recommendations

Remediation: Add a requirement in `createTokenId()` to ensure the expiry timestamp is strictly greater than the market entry end timestamp.

Resolution: In commit `147faa0`, a require statement is introduced to check if the `expiry` exceeds `market_entry_endrequire(expiry > market_entry_end)`.

F-2026-14815 - Inactive Buyers Remain in `_buyers` Indefinitely - Info

Description:

The `CustodyVault` contract maintains an `EnumerableSet.Bytes32Set` named `_buyers` to track all users who have ever held a balance or interacted with the vault. This set is used to support view functions like `buyerCount()` and `buyerAt()`. The contract adds buyers to the `_buyers` registry when they receive tokens (via `allocate()`, `deposit()`, or `internalTransfer()`) but never removes them, even if their balance drops to zero across all token IDs. This leads to a continuously growing registry that includes inactive users, potentially increasing the gas cost for off-chain indexers or future on-chain logic that iterates over this set.

Currently, `_buyers.add(buyerHash)` is called in multiple functions (`allocate()`, `internalTransfer()`, `depositFor()`, etc.). However, there is no corresponding `_buyers.remove(buyerHash)` logic in functions that reduce balances, such as `retire`, `withdrawToExternal()`, or `internalTransfer()` (for the sender).

As a result, a buyer who sells or retires all their tokens remains in the `_buyers` list forever. While this serves as a historical record, it conflates "current holders" with "historical users," making the `buyerCount()` metric misleading as a measure of active participation. This can bloat the storage and increase the consumption.

Assets:

- `CustodyVault.sol` [<https://github.com/S3-Markets/s3block>]

Status:

Fixed

Classification

Impact Rate:	1/5
Likelihood Rate:	1/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Info

Recommendations

Remediation: Implement a mechanism to remove buyers when they no longer hold any assets.

Resolution:

In commit `8b9c8f7`, the `_removeBuyerIfInactive(fromHash)` function is introduced to check if the buyer balance is zero and if so to remove buyer hash from mapping `_buyers.remove(buyerHash)`.

Disclaimers

Hacken Disclaimer

The smart contracts given for audit have been analyzed based on best industry practices at the time of the writing of this report, with cybersecurity vulnerabilities and issues in smart contract source code, the details of which are disclosed in this report (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

The report contains no statements or warranties on the identification of all vulnerabilities and security of the code. The report covers the code submitted and reviewed, so it may not be relevant after any modifications. Do not consider this report as a final and sufficient assessment regarding the utility and safety of the code, bug-free status, or any other contract statements.

While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only — we recommend proceeding with several independent audits and a public bug bounty program to ensure the security of smart contracts.

English is the original language of the report. The Consultant is not responsible for the correctness of the translated versions.

As part of Hacken's ongoing quality assurance process, we may conduct re-audits of select projects. These re-audits are performed independently from the original audit and are intended solely for internal quality control and improvement. Updated reports resulting from such re-audits will be shared privately with the respective clients and may be published on the Hacken website only with their explicit consent.

The sole authoritative source for finalized and most up-to-date versions of all reports remains the Audits section at <https://hacken.io/audits/>.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have vulnerabilities that can lead to hacks. Thus, the Consultant cannot guarantee the explicit security of the audited smart contracts.

Appendix 1. Definitions

Severities

When auditing smart contracts, Hacken is using a risk-based approach that considers **Likelihood**, **Impact**, **Exploitability** and **Complexity** metrics to evaluate findings and score severities.

Reference on how risk scoring is done is available through the repository in our Github organization:

[hknio/severity-formula](https://github.com/hacken/severity-formula)

Severity	Description
Critical	Critical vulnerabilities are usually straightforward to exploit and can lead to the loss of user funds or contract state manipulation.
High	High vulnerabilities are usually harder to exploit, requiring specific conditions, or have a more limited scope, but can still lead to the loss of user funds or contract state manipulation.
Medium	Medium vulnerabilities are usually limited to state manipulations and, in most cases, cannot lead to asset loss. Contradictions and requirements violations. Major deviations from best practices are also in this category.
Low	Major deviations from best practices or major Gas inefficiency. These issues will not have a significant impact on code execution.

Potential Risks

The "Potential Risks" section identifies issues that are not direct security vulnerabilities but could still affect the project's performance, reliability, or user trust. These risks arise from design choices, architectural decisions, or operational practices that, while not immediately exploitable, may lead to problems under certain conditions. Additionally, potential risks can impact the quality of the audit itself, as they may involve external factors or components beyond the scope of the audit, leading to incomplete assessments or oversight of key areas. This section aims to provide a broader perspective on factors that could affect the project's long-term security, functionality, and the comprehensiveness of the audit findings.

Appendix 2. Scope

The scope of the project includes the following smart contracts from the provided repository:

Scope Details	
Repository	https://github.com/S3-Markets/s3block
Initial Commit	1df2037
Final Commit	00fc907
Whitepaper	N/A
Requirements	Readme.md
Technical Requirements	Readme.md

Asset	Type
CustodyVault.sol [https://github.com/S3-Markets/s3block]	Smart Contract
EAC1155.sol [https://github.com/S3-Markets/s3block]	Smart Contract
TransferGate.sol [https://github.com/S3-Markets/s3block]	Smart Contract
UUPSPProxy.sol [https://github.com/S3-Markets/s3block]	Smart Contract

Appendix 3. Additional Valuables

Additional Recommendations

The smart contracts in the scope of this audit could benefit from the introduction of automatic emergency actions for critical activities, such as unauthorized operations like ownership changes or proxy upgrades, as well as unexpected fund manipulations, including large withdrawals or minting events. Adding such mechanisms would enable the protocol to react automatically to unusual activity, ensuring that the contract remains secure and functions as intended.

To improve functionality, these emergency actions could be designed to trigger under specific conditions, such as:

- Detecting changes to ownership or critical permissions.
- Monitoring large or unexpected transactions and minting events.
- Pausing operations when irregularities are identified.

These enhancements would provide an added layer of security, making the contract more robust and better equipped to handle unexpected situations while maintaining smooth operations.

Frameworks and Methodologies

This security assessment was conducted in alignment with recognised penetration testing standards, methodologies and guidelines, including the [NIST SP 800-115 – Technical Guide to Information Security Testing and Assessment](#), and the [Penetration Testing Execution Standard \(PTES\)](#). These assets provide a structured foundation for planning, executing, and documenting technical evaluations such as vulnerability assessments, exploitation activities, and security code reviews. Hacken's internal penetration testing methodology extends these principles to Web2 and Web3 environments to ensure consistency, repeatability, and verifiable outcomes.